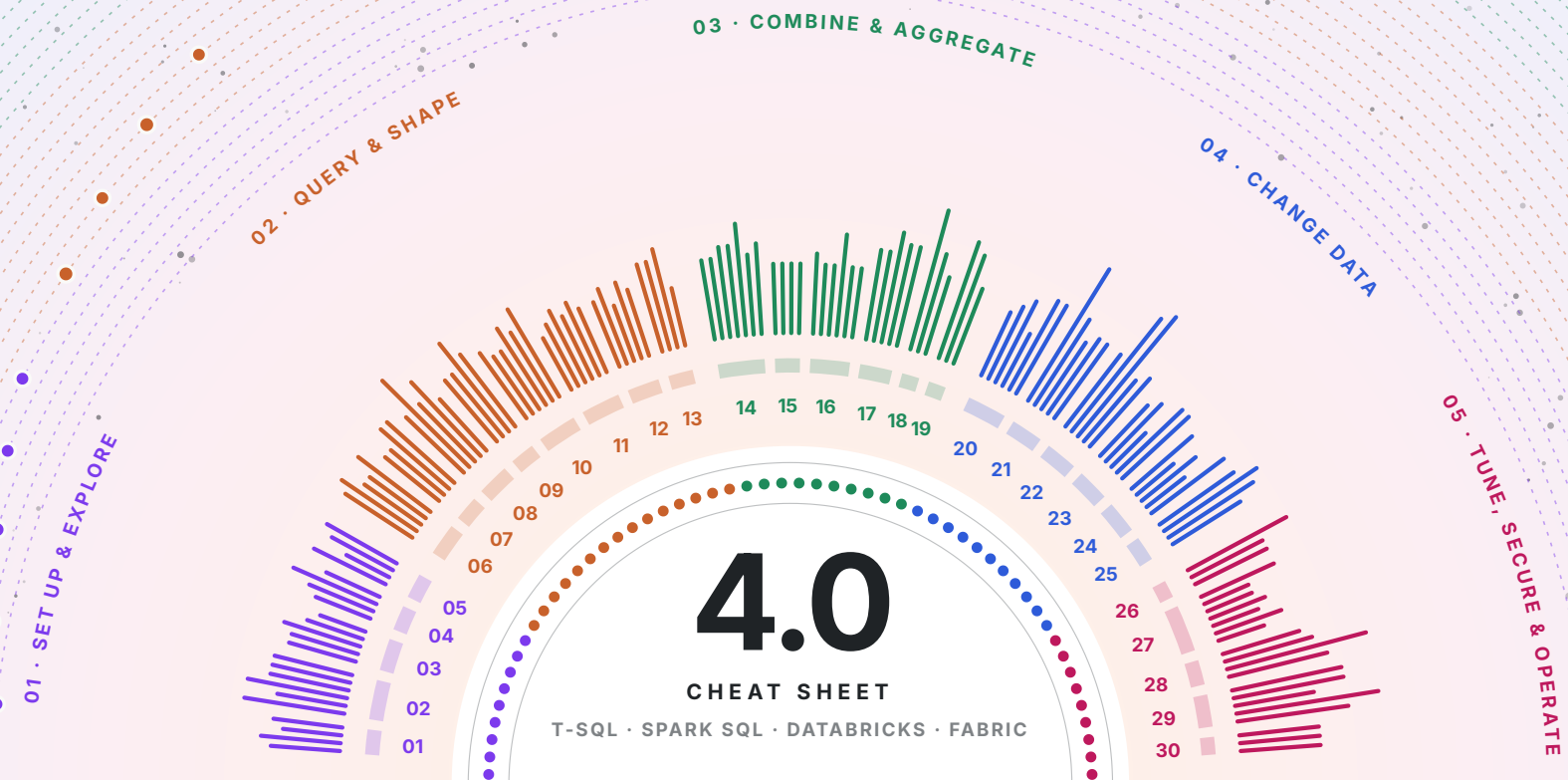


SQL Code.

142 patterns for Databricks and Fabric, from a first SELECT to MERGE, time travel and security, in T-SQL and Spark SQL.



01 Set up & Explore	02 Query & Shape	03 Combine & Aggregate	04 Change Data	05 Tune, Secure & Operate
01 Context & Names	06 Select & Filter	14 Joins	20 Insert, Update, Delete	26 Query Plans
02 Create & Alter	07 NULL Handling	15 Set Operations	21 MERGE	27 Performance & Maintenance
03 Inspect & Metadata	08 Sorting & Paging	16 Aggregations	22 SCD Type 1 & 2	28 Security
04 Data Types & Casting	09 Conditional Logic	17 Window Functions	23 Loading Data	29 Scripting & Errors
05 Variables & Parameters	10 Strings	18 CTEs & Subqueries	24 Time Travel & History	30 Notebooks & Pipelines
	11 Dates & Times	19 Pivot & Unpivot	25 Views & Materialised Views	
	12 JSON & Semi-structured			
	13 Arrays & Maps			

Appendix A T-SQL to Spark SQL

61

Appendix B Common errors

63

Notes What changed

66

BY

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn

COMPANION

The SQL Practitioner's Handbook 4.0

Coming next: how Fabric and Databricks plan and run your SQL.



WHY THIS EXISTS

Keep it open while you query.

SQL on Fabric and Databricks means two dialects. T-SQL runs in the Fabric Warehouse; Spark SQL runs in Databricks, Fabric notebooks and any lakehouse. Most of the language is shared, and the differences sit in the details you stop checking. This sheet puts the everyday patterns side by side, on one dataset, so you stop translating from memory.

01 Know which engine runs the query

A Fabric Warehouse and its SQL analytics endpoint speak T-SQL. A lakehouse notebook, a Databricks SQL warehouse and a Databricks notebook speak Spark SQL. Check the engine before you debug the query.

02 Keep it beside your editor

Find the task, adapt the pattern to your own tables, run it, and compare what you get with what the sheet shows.

03 Write it first, then ask the AI

Assistants write SQL fast, and they mix dialects without warning: a `TOP` in Spark, a `DATEDIFF` with its arguments the wrong way round. You cannot review a query you could not have written. Write it first and ask afterwards.

04 One dataset, messy on purpose

Every example queries the same tables, `employees` and `departments`. They carry what real data carries: a duplicate employee, 2 missing salaries, a salary of -5000 and a contractor in a department that does not exist. When a pattern trips over them, that is the lesson.

05 Practise from memory

Pick a section, close the sheet, and write its queries against your own tables. Then check. Do it again next week. Reading a query feels like knowing it; writing it from memory is knowing it.

HOW TO READ A SECTION

Pattern and code

One query, used where both dialects agree or only one has the feature.

T-SQL · Spark SQL

Side-by-side panels where the dialects differ: Fabric Warehouse first, Spark SQL second.

Spark 4.1+

A badge gives the minimum version or the platform a pattern needs.

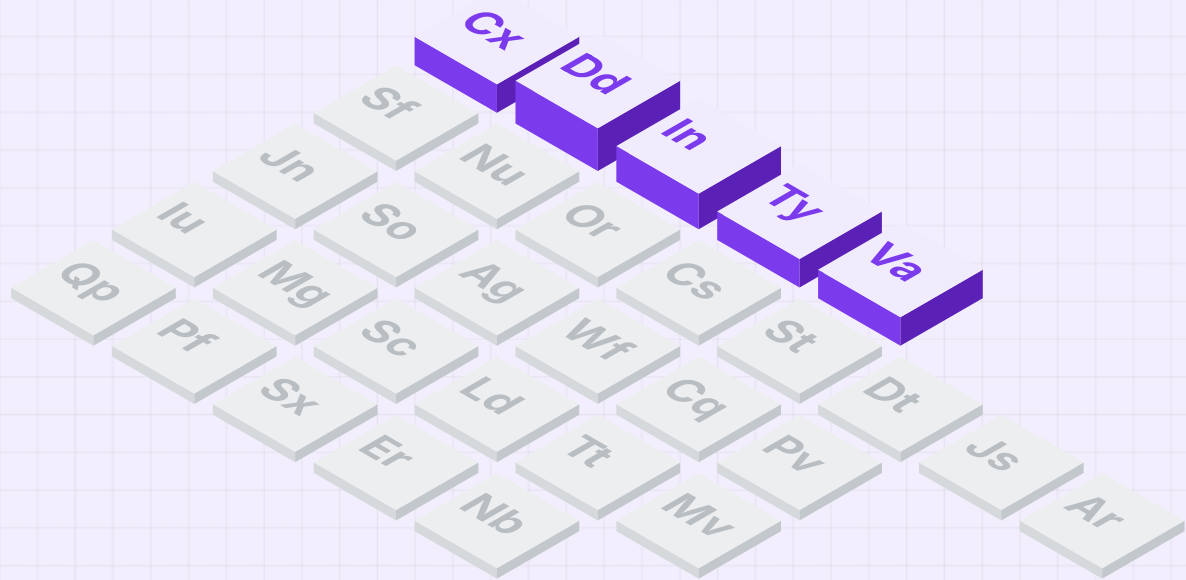
Watch out

What goes wrong in practice, and the fix, kept short.

Sanjay Chandra

Every pattern open-source Spark can run was run on Spark 4.1 against this dataset, and the T-SQL was parsed. Delta, Unity Catalog and Fabric features were checked against the documentation.

CHAPTER 01 OF 05



Set up & Explore.

Point your queries at the right place, create and change tables, read the metadata, get the types right and pass values in safely.

05

SECTIONS

23

PATTERNS

09

WATCH OUT

IN THIS CHAPTER

01	Context & Names	04
02	Create & Alter	05
03	Inspect & Metadata	07
04	Data Types & Casting	09
05	Variables & Parameters	11

SECTION 01 · CONTEXT & NAMES

Context & Names.

04
PATTERNS

02
WATCH OUT

Which warehouse, catalog and schema a query runs against, and how to name tables across them.

01 Point queries at a schema

Spark keeps a current schema per session; T-SQL names the schema in each query

T-SQL · FABRIC WAREHOUSE

```
-- a connection opens one warehouse;
-- dbo is the default schema
SELECT TOP 5 * FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
USE hr;
SELECT * FROM employees LIMIT 5;
```

02 Name tables in another item

Fabric: warehouse.schema.table across items in one workspace. Unity Catalog: catalog.schema.table

T-SQL · FABRIC WAREHOUSE

```
SELECT e.employee_name, d.department_name
FROM HRWarehouse.dbo.employees AS e
JOIN RefLakehouse.dbo.departments AS d
ON d.department_id = e.department_id;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT e.employee_name, d.department_name
FROM main.hr.employees AS e
JOIN main.ref.departments AS d
ON d.department_id = e.department_id;
```

03 Switch catalog on

Databricks **Databricks**

```
USE CATALOG main;
USE SCHEMA hr;
```

SECTION 01 · CONTEXT & NAMES

CONTINUED

04 Check where you are

T-SQL · FABRIC WAREHOUSE

```
SELECT DB_NAME()      AS warehouse,
       SCHEMA_NAME()  AS default_schema,
       SUSER_SNAME()  AS login;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT current_catalog() AS catalog,
       current_schema()  AS schema,
       current_user()     AS login;
```

WATCH OUT

SQL analytics endpoint

Read-only. INSERT, UPDATE and DELETE fail there; write through the Warehouse or Spark.

Case matters

A Fabric Warehouse uses a case-sensitive collation by default, so 'london' does not match 'London'.

SECTION 02 · CREATE & ALTER

Create & Alter.

06

PATTERNS

02

WATCH OUT

Create tables, copy them from a query, add identity keys and constraints, and change columns later.

01 Create a table Tables are Delta by default on Databricks and in a Fabric lakehouse

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.employees (
  employee_id INT NOT NULL,
  employee_name VARCHAR(100),
  hire_date DATE,
  salary DECIMAL(12,2)
);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE TABLE employees_new (
  employee_id INT NOT NULL,
  employee_name STRING,
  hire_date DATE,
  salary DECIMAL(12,2)
);
```

02 Create a table from a query CTAS copies the column types from the query; add constraints afterwards

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.eng AS
SELECT * FROM dbo.employees
WHERE department_id = 101;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE TABLE eng AS
SELECT * FROM employees
WHERE department id = 101;
```

SECTION 02 · CREATE & ALTER

CONTINUED

03 Add an identity key [Delta on Databricks](#)

Fabric: BIGINT only, no seed or increment. Values are unique, not gap-free

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.dim_department (
  department_key BIGINT IDENTITY,
  department_id INT,
  department_name VARCHAR(60)
);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE TABLE dim_department (
  department_key BIGINT
    GENERATED ALWAYS AS IDENTITY,
  department_id INT,
  department_name STRING
);
```

04 Add, rename and drop a column

Delta needs column mapping for RENAME and DROP: set delta.columnMapping.mode = name

T-SQL · FABRIC WAREHOUSE

```
ALTER TABLE dbo.employees ADD email VARCHAR(200);
EXEC sp_rename 'dbo.employees.email',
               'work_email', 'COLUMN';
ALTER TABLE dbo.employees DROP COLUMN work_email;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
ALTER TABLE employees ADD COLUMN email2 STRING;
ALTER TABLE employees
  RENAME COLUMN email2 TO work_email;
ALTER TABLE employees DROP COLUMN work_email;
```

05 Session-scoped scratch table

The distributed #temp form is the one Microsoft recommends; a temp view stores no data

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE #eng (
  employee_id INT,
  salary DECIMAL(12,2)
) WITH (DISTRIBUTION = ROUND_ROBIN);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE OR REPLACE TEMP VIEW eng_v AS
SELECT employee_id, salary
FROM employees
WHERE department_id = 101;
```

SECTION 02 · CREATE & ALTER

CONTINUED

06 Add a constraint Fabric keys are metadata only. A Delta CHECK constraint is enforced: a bad write fails

T-SQL · FABRIC WAREHOUSE

```
ALTER TABLE dbo.employees
ADD CONSTRAINT pk_employees
PRIMARY KEY NONCLUSTERED (employee_id)
NOT ENFORCED;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
ALTER TABLE employees
ADD CONSTRAINT salary_positive
CHECK (salary > 0);
```

WATCH OUT

NOT ENFORCED

A Fabric primary key does not stop duplicates. Check for them in the load, because nothing else will.

DROP TABLE

Dropping a managed table deletes its data. External tables keep their files.

SECTION 03 · INSPECT & METADATA

Inspect & Metadata.

05

PATTERNS

01

WATCH OUT

List tables, read column types and table details, and look at a few rows before you write a query.

01 List tables

T-SQL · FABRIC WAREHOUSE

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
ORDER BY TABLE_SCHEMA, TABLE_NAME;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SHOW TABLES;
```

SECTION 03 · INSPECT & METADATA

CONTINUED

02 Describe the columns

T-SQL · FABRIC WAREHOUSE

```
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'employees';
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
DESCRIBE TABLE employees;
```

03 Show how a table is defined Spark prints the full CREATE statement; T-SQL reads the catalog views

T-SQL · FABRIC WAREHOUSE

```
SELECT c.name, TYPE_NAME(c.user_type_id) AS type,
       c.max_length, c.is_nullable
FROM sys.columns AS c
WHERE c.object_id = OBJECT_ID('dbo.employees');
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SHOW CREATE TABLE employees_new;
```

04 Delta table details and history Delta

```
DESCRIBE DETAIL employees; -- files, size, location
DESCRIBE HISTORY employees; -- every commit
```

05 Look at a sample TOP and LIMIT without ORDER BY return whichever rows arrive first

T-SQL · FABRIC WAREHOUSE

```
SELECT TOP 10 * FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT * FROM employees TABLESAMPLE (10 PERCENT);
```

WATCH OUT

SELECT *

Fine for a look. In a view or a job it breaks when a column is added or renamed; list the columns.

SECTION 04 · DATA TYPES & CASTING

Data Types & Casting.

04
PATTERNS

02
WATCH OUT

Map the types between the two dialects, cast without failing on bad input, and divide without errors.

TYPE MAP

FABRIC WAREHOUSE	SPARK SQL	NOTES
<code>varchar(n), char(n)</code>	<code>STRING</code>	No nvarchar in Fabric tables; varchar is UTF-8
<code>int, bigint, smallint</code>	<code>INT, BIGINT, SMALLINT</code>	No tinyint in Fabric tables: use smallint
<code>decimal(p,s)</code>	<code>DECIMAL(p,s)</code>	No money in Fabric tables: use decimal
<code>float, real</code>	<code>DOUBLE, FLOAT</code>	
<code>bit</code>	<code>BOOLEAN</code>	
<code>date</code>	<code>DATE</code>	
<code>datetime2(6)</code>	<code>TIMESTAMP</code>	No datetime or datetimeoffset in Fabric tables; 6 digits max
<code>varbinary(n)</code>	<code>BINARY</code>	
<code>varchar(max)</code> holding JSON	<code>VARIANT</code> or <code>STRING</code>	No json type in Fabric Warehouse

01 Cast without failing

T-SQL · FABRIC WAREHOUSE

```
SELECT TRY_CAST('12a' AS INT) AS bad_int;
-- NULL instead of an error
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT try_cast('12a' AS INT) AS bad_int;
-- NULL instead of an error
```

SECTION 04 · DATA TYPES & CASTING

CONTINUED

02 Text to a date

T-SQL · FABRIC WAREHOUSE

```
SELECT TRY_CAST('2026-09-30' AS date) AS d;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT to_date('30/09/2026', 'dd/MM/yyyy') AS d,
       try_to_timestamp('2026-09-30 14:05',
                        'yyyy-MM-dd HH:mm') AS ts;
```

03 Divide without an error

T-SQL · FABRIC WAREHOUSE

```
SELECT department_id,
       SUM(salary) / NULLIF(COUNT(salary), 0)
       AS avg_salary
FROM   dbo.employees
GROUP BY department_id;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT department_id,
       try_divide(SUM(salary), COUNT(salary))
       AS avg_salary
FROM   employees
GROUP BY department_id;
```

04 Check a column's type

Spark SQL

```
SELECT typeof(salary), typeof(hire_date)
FROM   employees LIMIT 1;
```

WATCH OUT

ANSI mode

Spark 4 and DBR 17+ raise `CAST_INVALID_INPUT` and overflow errors instead of returning NULL. Use `try_` functions where bad input is expected.

Decimal

Dividing two decimals can round early. Cast one side to a wider scale first.

SECTION 05 · VARIABLES & PARAMETERS

Variables & Parameters.

04
PATTERNS

02
WATCH OUT

Hold values in variables, pass parameters in safely, and build table names without string concatenation.

01 Declare a variable and use it Spark 4.0+

T-SQL · FABRIC WAREHOUSE

```
DECLARE @dept INT = 101;
SELECT employee_name
FROM dbo.employees
WHERE department_id = @dept;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
DECLARE VARIABLE dept INT DEFAULT 101;
SELECT employee_name
FROM employees
WHERE department_id = dept;
```

02 Change its value

T-SQL · FABRIC WAREHOUSE

```
SET @dept = 102;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SET VAR dept = 102;
```

03 Use a parameter marker Spark SQL

Databricks SQL editor shows :min_salary as an input box; spark.sql() fills it from args

```
SELECT employee_name, salary
FROM employees
WHERE salary > :min_salary;
```

SECTION 05 · VARIABLES & PARAMETERS

CONTINUED

04 Table name from a variable Spark 4.0+

IDENTIFIER() treats the value as a name, so it cannot inject SQL

T-SQL · FABRIC WAREHOUSE

```
DECLARE @tbl VARCHAR(128) = 'employees';
DECLARE @sql VARCHAR(400) =
    'SELECT COUNT(*) FROM dbo.' + QUOTENAME(@tbl);
EXEC (@sql);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
DECLARE VARIABLE tbl STRING DEFAULT 'employees';
SELECT count(*) FROM IDENTIFIER(tbl);
```

WATCH OUT

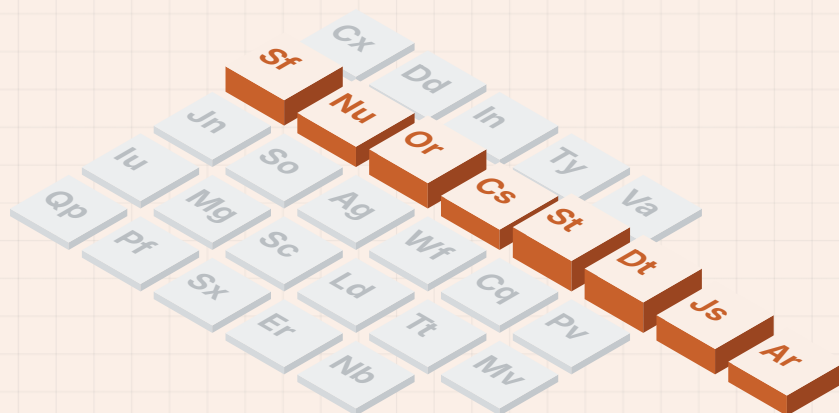
Batch scope

A T-SQL variable dies at the end of the batch. A Spark session variable lives until the session ends.

Never concatenate input

Pass values as parameters. Only table and column names need QUOTENAME or IDENTIFIER().

CHAPTER 02 OF 05



Query & Shape.

Filter and sort rows, deal with NULL, branch with CASE, and clean up strings, dates, JSON and arrays.

08

SECTIONS

39

PATTERNS

15

WATCH OUT

IN THIS CHAPTER

06	Select & Filter	14
07	NULL Handling	16
08	Sorting & Paging	18
09	Conditional Logic	19
10	Strings	20
11	Dates & Times	23
12	JSON & Semi-structured	25
13	Arrays & Maps	27

SECTION 06 · SELECT & FILTER

Select & Filter.

05

PATTERNS

01

WATCH OUT

Pick rows and columns, match patterns, and reuse a calculated column without repeating it.

THE DATA EVERY PATTERN QUERIES

DEPARTMENTS · ALL 10 ROWS

department_id	department_name	location	region
101	Engineering	London	EMEA
102	Data & Analytics	Singapore	APAC
103	Sales	New York	AMER
104	Human Resources	London	EMEA
105	Finance	New York	AMER
106	Product Management	Singapore	APAC
107	Cloud Infrastructure	Sydney	APAC
108	Cybersecurity	New York	AMER
109	Legal & Compliance	London	EMEA
110	Executive Office	New York	AMER

- 20 employee rows, 19 unique employee_id values (1007 appears twice, with two salaries)
- 10 departments. Cybersecurity (108) has no employees; one contractor sits in department 111, which does not exist
- 2 missing salaries, 1 of -5000, 1 missing hire date
- A trailing space in one name and an upper-case email, for the string patterns
- manager_id links each employee to a manager, for the hierarchy patterns

EMPLOYEES · ALL 20 ROWS

employee_id	employee_name	job_title	department_id	manager_id	hire_date	salary	email
1000	Grace Okafor	Chief Executive	110	null	2012-05-14	310000	grace.okafor@corp.com
1001	Danielle Johnson	Engineering Manager	101	1000	2016-02-01	142000	danielle.johnson@corp.com
1002	Jeremy Roberts	Senior Engineer	101	1001	2018-07-23	118000	jeremy.roberts@corp.com
1003	Priya Nair	Data Engineer	102	1006	2021-03-15	96000	priya.nair@corp.com
1004	Robert Ramirez	Data Analyst	102	1006	2023-03-19	92000	robert.ramirez@corp.com
1005	Martha Warner	Account Executive	103	1009	2019-11-04	88000	martha.warner@corp.com
1006	Aisha Bello	Head of Data	102	1000	2017-09-12	150000	aisha.bello@corp.com
1007	Mark Lin	Financial Analyst	105	1010	2022-01-10	79000	mark.lin@corp.com
1007	Mark Lin	Financial Analyst	105	1010	2022-01-10	82000	mark.lin@corp.com
1008	Tom Becker	Junior Engineer	101	1002	2024-08-05	null	tom.becker@corp.com
1009	Elena Rossi	Sales Director	103	1000	2015-06-01	165000	elena.rossi@corp.com
1010	Samuel Otieno	Finance Manager	105	1000	2014-10-20	138000	samuel.otieno@corp.com
1011	April Williams	Associate PM	106	1013	2020-06-08	-5000	april.williams@corp.com
1012	Wei Zhang	Cloud Architect	107	1001	2022-08-22	129000	wei.zhang@corp.com
1013	Lucas Moreau	Product Director	106	1000	2016-12-05	158000	lucas.moreau@corp.com
1014	Hannah Klein	HR Partner	104	1000	null	71000	hannah.klein@corp.com
1015	Carlos Mendes	Sales Executive	103	1009	2023-05-30	67000	CARLOS.MENDES@CORP.COM
1016	Yuki Tanaka	Legal Counsel	109	1000	2019-02-18	121000	yuki.tanaka@corp.com
1017	Omar Haddad	Data Engineer	102	1006	2024-02-12	null	omar.haddad@corp.com
1018	Nina Petrova	Contractor	111	1001	2025-01-06	60000	nina.petrova@corp.com

Tinted rows: employee 1007 appears twice. Coloured values: the messy ones the patterns trip over. marks a trailing space.

SECTION 06 · SELECT & FILTER

CONTINUED

PATTERN

CODE

01 Filter rows

```
SELECT employee_name, salary
FROM employees
WHERE department_id = 101
AND salary > 100000;
```

02 IN, BETWEEN and LIKE

```
SELECT employee_name, hire_date
FROM employees
WHERE department_id IN (101, 102)
AND hire_date BETWEEN '2020-01-01' AND '2024-12-31'
AND employee_name LIKE 'P%';
```

03 Match text ignoring case

Needed in Fabric because the default collation is case-sensitive

T-SQL · FABRIC WAREHOUSE

```
SELECT employee_name
FROM dbo.employees
WHERE LOWER(employee_name) LIKE 'p%';
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT employee_name
FROM employees
WHERE employee_name ILIKE 'p%';
```

04 Distinct values

```
SELECT DISTINCT department_id
FROM employees
ORDER BY department_id;
```

05 Reuse a calculated column

Spark lets a later column use an alias defined earlier in the same SELECT

T-SQL · FABRIC WAREHOUSE

```
SELECT e.employee_name, x.new_salary,
       x.new_salary - e.salary AS raise
FROM dbo.employees AS e
CROSS APPLY (SELECT e.salary * 1.1
              AS new_salary) AS x;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT employee_name,
       salary * 1.1 AS new_salary,
       new_salary - salary AS raise
FROM employees;
```

SECTION 06 · SELECT & FILTER

CONTINUED

WATCH OUT

`!=` skips NULL

`WHERE salary <> 0` also drops the two rows with no salary. Add `OR salary IS NULL` when you mean to keep them.

SECTION 07 · NULL HANDLING

NULL Handling.

04
PATTERNS

02
WATCH OUT

Count, compare, replace and sort NULL on purpose, instead of finding out from a wrong total.

RESULT · WHAT EACH COUNT COUNTS

```
SELECT COUNT(*)           AS all_rows,
       COUNT(salary)      AS with_salary,
       COUNT(DISTINCT employee_id) AS unique_ids
FROM employees;
```

all_rows	with_salary	unique_ids
20	18	19

01 Replace NULL with a default

Same on both. T-SQL also has ISNULL; Spark also has nvl and ifnull

```
SELECT employee_name,
       COALESCE(salary, 0) AS salary
FROM employees;
```

02 Turn a sentinel into NULL

```
SELECT employee_name,
       NULLIF(salary, -5000) AS salary
FROM employees;
```

RESULT · AVERAGES IGNORE NULL

```
SELECT AVG(salary)           AS avg_known,
       AVG(COALESCE(salary, 0)) AS avg_as_zero
FROM employees;
```

avg_known	avg_as_zero
114500.0	103050.0

Two different answers from the same column: decide which one the report means

SECTION 07 • NULL HANDLING

CONTINUED

03 Compare values that may be NULL Spark also accepts IS NOT DISTINCT FROM

T-SQL • FABRIC WAREHOUSE

```
SELECT a.employee_name, b.employee_name
FROM dbo.employees AS a
JOIN dbo.employees AS b
  ON a.manager_id IS NOT DISTINCT FROM b.manager_id
 AND a.employee_id < b.employee_id;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT a.employee_name, b.employee_name
FROM employees AS a
JOIN employees AS b
  ON a.manager_id <=> b.manager_id
 AND a.employee_id < b.employee_id;
```

04 Put NULL last when sorting

T-SQL • FABRIC WAREHOUSE

```
SELECT employee_name, salary
FROM dbo.employees
ORDER BY CASE WHEN salary IS NULL
              THEN 1 ELSE 0 END,
         salary DESC;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT employee_name, salary
FROM employees
ORDER BY salary DESC NULLS LAST;
```

WATCH OUT

= NULL

Never true, not even for NULL. Use IS NULL.

NOT IN

If the subquery returns one NULL, **NOT IN** returns no rows at all. Use NOT EXISTS (Joins, section 14).

SECTION 08 · SORTING & PAGING

Sorting & Paging.

05
PATTERNS

02
WATCH OUT

Take the top rows, sort on several keys or a custom order, and page through a result.

01 Top N rows

T-SQL · FABRIC WAREHOUSE

```
SELECT TOP 5 employee_name, salary
FROM dbo.employees
ORDER BY salary DESC;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT employee_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

02 Page through a result

Works the same on both engines

```
WITH ranked AS (
    SELECT employee_id, employee_name,
           ROW_NUMBER() OVER (ORDER BY employee_id) AS rn
    FROM employees
)
SELECT employee_id, employee_name
FROM ranked
WHERE rn BETWEEN 11 AND 15;
```

03 LIMIT with OFFSET

Spark SQL

```
SELECT employee_id, employee_name
FROM employees
ORDER BY employee_id
LIMIT 5 OFFSET 10;
```

04 Custom sort order

```
SELECT department_name, region
FROM departments
ORDER BY CASE region WHEN 'EMEA' THEN 1
                  WHEN 'AMER' THEN 2
                  ELSE 3 END,
         department_name;
```

05 Sort by every column

Works on both engines

```
SELECT department_id, job_title
FROM employees
ORDER BY ALL;
```

WATCH OUT

ORDER BY in a view

Sorting inside a view or CTE does not order the final result. Sort in the outer query.

Ties

With duplicate sort keys, TOP and LIMIT can return different rows on each run. Add a unique column last.

SECTION 09 · CONDITIONAL LOGIC

Conditional Logic.

04
PATTERNS

02
WATCH OUT

Branch with CASE, pick a value inline, and count conditionally in one pass.

RESULT · BAND SALARIES WITH CASE

```
SELECT employee_name, salary,
       CASE WHEN salary >= 150000 THEN 'high'
            WHEN salary >= 90000  THEN 'mid'
            WHEN salary > 0       THEN 'base'
            ELSE 'check'
       END AS band
FROM employees
WHERE department_id IN (102, 106);
```

employee_name	salary	band
Priya Nair	96000	mid
Robert Ramirez	92000	mid
Aisha Bello	150000	high
April Williams	-5000	check
Lucas Moreau	158000	high
Omar Haddad	null	check

01 Pick a value inline

T-SQL · FABRIC WAREHOUSE

```
SELECT employee_name,
       IIF(salary > 100000, 'senior', 'other')
       AS level
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT employee_name,
       if(salary > 100000, 'senior', 'other')
       AS level
FROM employees;
```

02 Largest or smallest of several values

```
SELECT employee_name,
       GREATEST(salary, 50000) AS floored,
       LEAST(salary, 200000)  AS capped
FROM employees;
```

03 Count several conditions in one pass

```
SELECT department_id,
       COUNT(CASE WHEN salary > 100000 THEN 1 END)
       AS high_earners,
       SUM(CASE WHEN hire_date >= '2023-01-01'
                THEN 1 ELSE 0 END) AS recent_hires
FROM employees
GROUP BY department_id;
```

SECTION 09 · CONDITIONAL LOGIC

CONTINUED

PATTERN

CODE

04 The FILTER clause

Spark SQL

```
SELECT department_id,
       COUNT(*) FILTER (WHERE salary > 100000)
       AS high_earners
FROM employees
GROUP BY department_id;
```

WATCH OUT

Order of WHEN

CASE stops at the first true branch. Put the narrowest condition first.

Missing ELSE

Without ELSE, unmatched rows get NULL. The -5000 salary lands in `check` above only because ELSE says so.

SECTION 10 · STRINGS

Strings.

06

PATTERNS

02

WATCH OUT

Trim, join, cut, split and aggregate text, and know where the two dialects disagree.

01 Trim, case and length LEN ignores trailing spaces, length counts them: 13 against 14 for 'Carlos Mendes '

T-SQL · FABRIC WAREHOUSE

```
SELECT TRIM(employee_name) AS name,
       UPPER(job_title)    AS title,
       LEN(employee_name)  AS n
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT trim(employee_name) AS name,
       upper(job_title)    AS title,
       length(employee_name) AS n
FROM employees;
```

SECTION 10 • STRINGS

CONTINUED

02 Join text, skipping NULL

T-SQL CONCAT treats NULL as empty; Spark concat and || return NULL if any part is NULL

T-SQL • FABRIC WAREHOUSE

```
SELECT CONCAT(employee_name, ' <',
              email, '>') AS contact
FROM dbo.employees;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT concat_ws(' ', employee_name,
                '<' || email || '>') AS contact
FROM employees;
```

03 Text before a character

T-SQL • FABRIC WAREHOUSE

```
SELECT LEFT(email,
            CHARINDEX('@', email) - 1) AS mailbox
FROM dbo.employees;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT substring_index(email, '@', 1)
       AS mailbox
FROM employees;
```

04 Regular expressions

Spark SQL

```
SELECT regexp_replace(employee_name, ' +$', '')
       AS clean_name,
       regexp_extract(email, '^[a-z]+\.\.', 1)
       AS first_name
FROM employees;
```

SECTION 10 • STRINGS

CONTINUED

05 Split into rows

T-SQL • FABRIC WAREHOUSE

```
SELECT e.employee_id, s.value AS word
FROM dbo.employees AS e
CROSS APPLY STRING_SPLIT(e.job_title, ' ')
AS s;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT employee_id, word
FROM employees
LATERAL VIEW explode(split(job_title, ' '))
AS word;
```

06 Aggregate into a list Spark 4.0+

T-SQL • FABRIC WAREHOUSE

```
SELECT department_id,
    STRING_AGG(employee_name, ', ')
    WITHIN GROUP (ORDER BY employee_name)
    AS names
FROM dbo.employees
GROUP BY department_id;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
SELECT department_id,
    string_agg(employee_name, ', ')
    WITHIN GROUP (ORDER BY employee_name)
    AS names
FROM employees
GROUP BY department_id;
```

WATCH OUT

Case-sensitive by default

Both engines compare text byte for byte unless a collation says otherwise. Normalise with LOWER or UPPER before joining on text.

Trailing spaces

Trim keys before you join or deduplicate: 'Carlos Mendes ' is not 'Carlos Mendes'.

SECTION 11 · DATES & TIMES

Dates & Times.

06
PATTERNS02
WATCH OUT

Get today, add and subtract, measure gaps, truncate, format and generate a calendar.

01 Today and now

T-SQL · FABRIC WAREHOUSE

```
SELECT CAST(GETDATE() AS date) AS today,  
        SYSUTCDATETIME() AS now_utc;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT current_date() AS today,  
        current_timestamp() AS now;
```

02 Add months and days

T-SQL · FABRIC WAREHOUSE

```
SELECT DATEADD(month, 6, hire_date) AS review,  
        DATEADD(day, 30, hire_date) AS probation  
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT add_months(hire_date, 6) AS review,  
        date_add(hire_date, 30) AS probation  
FROM employees;
```

SECTION 11 · DATES & TIMES

CONTINUED

03 Days between two dates The argument order is reversed: T-SQL (unit, start, end), Spark (end, start)

T-SQL · FABRIC WAREHOUSE

```
SELECT employee_name,
       DATEDIFF(day, hire_date, '2026-09-30')
       AS days_employed
FROM   dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT employee_name,
       datediff('2026-09-30', hire_date)
       AS days_employed
FROM   employees;
```

04 Truncate to the month Spark date_trunc returns a timestamp; trunc keeps a date

T-SQL · FABRIC WAREHOUSE

```
SELECT DATETRUNC(month, hire_date)
       AS hire_month
FROM   dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT trunc(hire_date, 'MM')
       AS hire_month
FROM   employees;
```

05 Year, quarter and day name

T-SQL · FABRIC WAREHOUSE

```
SELECT YEAR(hire_date)           AS y,
       DATEPART(quarter, hire_date) AS q,
       DATENAME(weekday, hire_date) AS dow
FROM   dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT year(hire_date)           AS y,
       quarter(hire_date)        AS q,
       date_format(hire_date, 'EEEE') AS dow
FROM   employees;
```

SECTION 11 · DATES & TIMES

CONTINUED

06 Generate a calendar

T-SQL · FABRIC WAREHOUSE

```
SELECT DATEADD(day, value, '2026-01-01')
      AS d
FROM GENERATE_SERIES(0, 364);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT explode(sequence(DATE'2026-01-01',
                        DATE'2026-12-31',
                        INTERVAL 1 DAY)) AS d;
```

WATCH OUT

Boundaries

T-SQL DATEDIFF counts boundaries crossed: 31 December to 1 January is 1 year.

Time zones

Fabric Warehouse returns UTC. Spark uses `spark.sql.session.timeZone`; set it once per job.

SECTION 12 · JSON & SEMI-STRUCTURED

JSON & Semi-structured.

05

PATTERNS

02

WATCH OUT

Read values out of JSON, store it in a VARIANT column, shred arrays into rows and build JSON back.

01 Read a value from JSON text

T-SQL · FABRIC WAREHOUSE

```
SELECT JSON_VALUE(payload, '$.device.os')
      AS os
FROM dbo.events;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT get_json_object(payload,
                        '$.device.os') AS os
FROM events;
```

SECTION 12 · JSON & SEMI-STRUCTURED

CONTINUED

PATTERN

CODE

02 Store JSON as VARIANT and query it

Spark 4.1+

VARIANT keeps the structure, so reads skip re-parsing the text

```
CREATE TABLE events_v (id BIGINT, payload VARIANT);
INSERT INTO events_v
SELECT 1, parse_json('{"device":{"os":"ios"}}');
SELECT payload:device.os::string AS os FROM events_v;
```

03 Shred a JSON array into rows

T-SQL · FABRIC WAREHOUSE

```
SELECT j.value AS skill
FROM OPENJSON(['sql","spark","dbt"]) AS j;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT explode(from_json(['sql","spark","dbt"],
                          'ARRAY<STRING>')) AS skill;
```

04 Build JSON from columns

T-SQL · FABRIC WAREHOUSE

```
SELECT JSON_OBJECT('id': employee_id,
                  'name': employee_name) AS doc
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT to_json(named_struct(
    'id', employee_id,
    'name', employee_name)) AS doc
FROM employees;
```

SECTION 12 · JSON & SEMI-STRUCTURED

CONTINUED

05 Check that text is valid JSON

T-SQL · FABRIC WAREHOUSE

```
SELECT payload
FROM dbo.events
WHERE ISJSON(payload) = 1;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT payload
FROM events
WHERE try_parse_json(payload) IS NOT NULL;
```

WATCH OUT

Fabric Warehouse

No json or VARIANT type: store JSON in varchar(max), which holds up to 16 MB.

Path typos

A wrong JSON path returns NULL, not an error. Check a few rows before trusting a count.

SECTION 13 · ARRAYS & MAPS

Arrays & Maps.

04

PATTERNS

02

WATCH OUT

Collect values into arrays, explode them back into rows, and transform them without a UDF.

PATTERN

CODE

01 Collect values into an array Spark SQL

```
SELECT department_id,
       collect_list(employee_name) AS names,
       collect_set(job_title)      AS titles
FROM employees
GROUP BY department_id;
```

02 Explode an array into rows Spark SQL

```
SELECT department_id, name
FROM (SELECT department_id,
              collect_list(employee_name) AS names
      FROM employees GROUP BY department_id)
LATERAL VIEW explode(names) AS name;
```

SECTION 13 · ARRAYS & MAPS

CONTINUED

PATTERN

CODE

03 Transform and filter inside an array

Spark SQL

Higher-order functions run inside the engine; no Python round trip

```
SELECT transform(array(1, 2, 3), x -> x * 10)
       AS tens,
       filter(array(5, -5, 10), x -> x > 0)
       AS positive,
       aggregate(array(1, 2, 3), 0, (a, x) -> a + x)
       AS total;
```

04 Maps and structs

Spark SQL

```
SELECT map('EMEA', 1, 'APAC', 2)['APAC']      AS apac,
       named_struct('id', 1, 'name', 'Priya').name AS n;
```

WATCH OUT

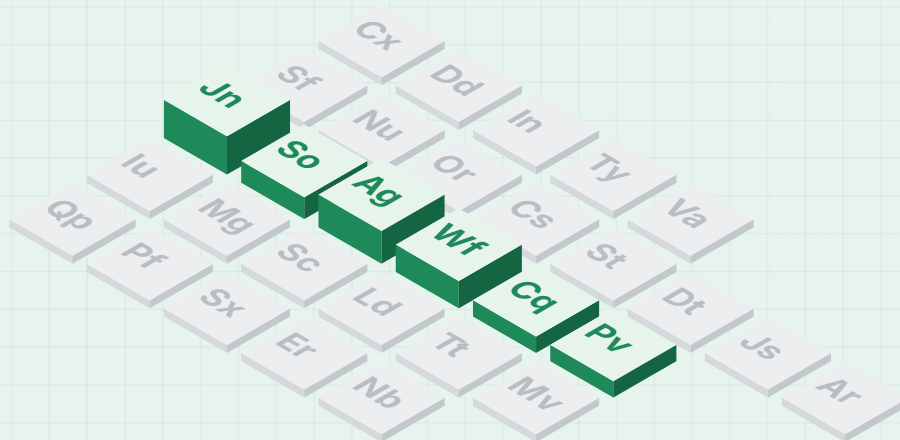
explode drops rows

A row whose array is empty or NULL disappears. Use `explode_outer` to keep it.

T-SQL

No array type. Split text with `STRING_SPLIT` or shred JSON with `OPENJSON` (sections 10 and 12).

CHAPTER 03 OF 05



Combine & Aggregate.

Join and stack tables, summarise with GROUP BY, rank and total with windows, build logic in CTEs, and pivot.

06

SECTIONS

28

PATTERNS

12

WATCH OUT

IN THIS CHAPTER

14	Joins	30
15	Set Operations	33
16	Aggregations	34
17	Window Functions	36
18	CTEs & Subqueries	38
19	Pivot & Unpivot	39

SECTION 14 · JOINS

Joins.

07
PATTERNS

03
WATCH OUT

Inner, outer, semi, anti, lateral and self joins, with the row counts they give on the dataset.

RESULT · ONE DATASET, FOUR JOIN TYPES

```
SELECT
  (SELECT COUNT(*) FROM employees e
   JOIN departments d USING (department_id))      AS inner_rows,
  (SELECT COUNT(*) FROM employees e
   LEFT JOIN departments d USING (department_id)) AS left_rows,
  (SELECT COUNT(*) FROM employees e
   RIGHT JOIN departments d USING (department_id)) AS right_rows,
  (SELECT COUNT(*) FROM employees e
   FULL JOIN departments d USING (department_id)) AS full_rows;
```

inner_rows	left_rows	right_rows	full_rows
19	20	20	21

Nina Petrova (department 111) only survives LEFT and FULL; Cybersecurity (108) only RIGHT and FULL

01 Inner join

```
SELECT e.employee_name, d.department_name
FROM employees AS e
JOIN departments AS d
  ON d.department_id = e.department_id;
```

02 Keep every employee
department_name is NULL
where nothing matched

```
SELECT e.employee_name, d.department_name
FROM employees AS e
LEFT JOIN departments AS d
  ON d.department_id = e.department_id;
```

SECTION 14 · JOINS

CONTINUED

03 Anti join: departments with nobody in them

Returns Cybersecurity. NOT EXISTS works on both engines

T-SQL · FABRIC WAREHOUSE

```
SELECT d.department_name
FROM dbo.departments AS d
WHERE NOT EXISTS (
    SELECT 1 FROM dbo.employees AS e
    WHERE e.department_id = d.department_id);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT d.department_name
FROM departments AS d
LEFT ANTI JOIN employees AS e
ON e.department_id = d.department_id;
```

04 Semi join: departments that have staff

T-SQL · FABRIC WAREHOUSE

```
SELECT d.department_name
FROM dbo.departments AS d
WHERE EXISTS (
    SELECT 1 FROM dbo.employees AS e
    WHERE e.department_id = d.department_id);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT d.department_name
FROM departments AS d
LEFT SEMI JOIN employees AS e
ON e.department_id = d.department_id;
```

SECTION 14 · JOINS

CONTINUED

05 Top earner per department with a lateral join

T-SQL · FABRIC WAREHOUSE

```
SELECT d.department_name, t.employee_name
FROM dbo.departments AS d
CROSS APPLY (
    SELECT TOP 1 e.employee_name
    FROM dbo.employees AS e
    WHERE e.department_id = d.department_id
    ORDER BY e.salary DESC) AS t;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT d.department_name, t.employee_name
FROM departments AS d
JOIN LATERAL (
    SELECT e.employee_name
    FROM employees AS e
    WHERE e.department_id = d.department_id
    ORDER BY e.salary DESC LIMIT 1) AS t;
```

06 Self join: employee and manager

```
SELECT e.employee_name, m.employee_name AS manager
FROM employees AS e
LEFT JOIN employees AS m
    ON m.employee_id = e.manager_id;
```

07 Force a broadcast join

Spark SQL

Ships the small table to every executor, so the big one is not shuffled

```
SELECT /*+ BROADCAST(d) */ e.employee_name,
      d.department_name
FROM employees AS e
JOIN departments AS d
    ON d.department_id = e.department_id;
```

WATCH OUT

Duplicate keys

Employee 1007 appears twice, so any join on employee_id returns two rows for him. Deduplicate first (section 17).

NOT IN and NULL

One NULL in the subquery and `NOT IN` returns nothing. `NOT EXISTS` is safe.

Mixed types

Joining INT to STRING forces a cast on every row and can drop matches. Fix the column types.

SECTION 15 • SET OPERATIONS

Set Operations.

04
PATTERNS

01
WATCH OUT

Stack results with UNION, and find what is in one set and not the other.

PATTERN	CODE
01 Stack two results UNION ALL keeps duplicates and skips a costly de-duplication step	<pre>SELECT department_id FROM employees UNION ALL SELECT department_id FROM departments;</pre>
02 Distinct values from both	<pre>SELECT department_id FROM employees UNION SELECT department_id FROM departments;</pre>
03 In the first set but not the second Returns 108. Spark also accepts MINUS	<pre>SELECT department_id FROM departments EXCEPT SELECT department_id FROM employees;</pre>
04 In both sets	<pre>SELECT department_id FROM departments INTERSECT SELECT department_id FROM employees;</pre>

WATCH OUT

By position

Set operators match columns by position, not by name. Swap two columns of the same type and the result is silently wrong.

SECTION 16 · AGGREGATIONS

Aggregations.

06
PATTERNS

02
WATCH OUT

Group and summarise, filter the groups, add subtotals, and get medians and approximate counts.

RESULT · HEADCOUNT AND PAYROLL PER DEPARTMENT

```
SELECT department_id,
       COUNT(*)      AS headcount,
       SUM(salary)   AS payroll,
       MAX(salary)   AS top_salary
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

<u>department_id</u>	headcount	payroll	top_salary
101	3	260000	142000
102	4	338000	150000
103	3	320000	165000
104	1	71000	71000
105	3	299000	138000
106	2	153000	158000
107	1	129000	129000
109	1	121000	121000
110	1	310000	310000
111	1	60000	60000

01 Filter groups with HAVING

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id
HAVING COUNT(*) >= 3;
```

02 Subtotals and a grand total

Rows with NULL in the grouped columns are the subtotals

```
SELECT department_id, job_title, SUM(salary) AS payroll
FROM employees
GROUP BY ROLLUP (department_id, job_title);
```

03 Several groupings in one pass

Works on both engines.
CUBE (a, b) gives every combination

```
SELECT department_id, job_title, SUM(salary) AS payroll
FROM employees
GROUP BY GROUPING SETS ((department_id), (job_title), ());
```

SECTION 16 · AGGREGATIONS

CONTINUED

04 Median salary per department

T-SQL computes PERCENTILE_CONT as a window only; Spark has it as an aggregate

T-SQL · FABRIC WAREHOUSE

```
SELECT DISTINCT department_id,
    PERCENTILE_CONT(0.5)
        WITHIN GROUP (ORDER BY salary)
        OVER (PARTITION BY department_id)
        AS median_salary
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT department_id,
    median(salary) AS median_salary
FROM employees
GROUP BY department_id;
```

05 Approximate distinct count Much cheaper than COUNT(DISTINCT) on large tables, within a few percent

T-SQL · FABRIC WAREHOUSE

```
SELECT APPROX_COUNT_DISTINCT(employee_id)
    AS approx_ids
FROM dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT approx_count_distinct(employee_id)
    AS approx_ids
FROM employees;
```

06 Group by every non-aggregate column

Works on both engines

```
SELECT department_id, job_title, COUNT(*) AS n
FROM employees
GROUP BY ALL;
```

WATCH OUT

Double counting

SUM over the raw table counts employee 1007 twice. Deduplicate before you aggregate.

WITH ROLLUP

The older GROUP BY a, b WITH ROLLUP form does not run in Fabric Warehouse. Write GROUP BY ROLLUP (a, b).

SECTION 17 · WINDOW FUNCTIONS

Window Functions.

05
PATTERNS

02
WATCH OUT

Rank, deduplicate, compare with the previous row, and compute running totals and shares.

RESULT · THREE RANKING FUNCTIONS ON A TIE

```
SELECT department_id, COUNT(*) AS headcount,
       ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS row_num,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS dense_rnk
FROM employees
GROUP BY department_id
ORDER BY row_num;
```

department_id	headcount	row_num	rnk	dense_rnk
102	4	1	1	1
101	3	2	2	2
103	3	3	2	2
105	3	4	2	2
106	2	5	5	3
110	1	6	6	4
107	1	7	6	4
104	1	8	6	4
109	1	9	6	4
111	1	10	6	4

Three departments tie on 3. ROW_NUMBER breaks the tie in no fixed order, RANK then skips to 5, DENSE_RANK does not skip

01 Top 2 earners per department Fabric Warehouse, Databricks

QUALIFY filters on a window without a subquery. Fabric Spark on Spark 4.1 needs the subquery form below

T-SQL · FABRIC WAREHOUSE

```
SELECT department_id, employee_name, salary
FROM dbo.employees
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC) <= 2;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT department_id, employee_name, salary
FROM employees
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC) <= 2;
```

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

RESULT · DEDUPLICATE: ONE ROW PER EMPLOYEE

```
SELECT employee_id, employee_name, salary
FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY employee_id
    ORDER BY salary DESC) AS rn
  FROM employees)
WHERE rn = 1 AND employee_id = 1007;
```

employee_id	employee_name	salary
1007	Mark Lin	82000

Keeps Mark Lin's higher salary. The subquery form works on both engines

02 Compare with the previous hire

```
SELECT department_id, employee_name, hire_date,
LAG(hire_date) OVER (
  PARTITION BY department_id
  ORDER BY hire_date) AS previous_hire
FROM employees;
```

03 Running total

```
SELECT department_id, employee_name, salary,
SUM(salary) OVER (
  PARTITION BY department_id
  ORDER BY hire_date
  ROWS BETWEEN UNBOUNDED PRECEDING
  AND CURRENT ROW) AS running_payroll
FROM employees;
```

04 Moving average of the last three

```
SELECT employee_name, hire_date,
AVG(salary) OVER (
  ORDER BY hire_date
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS moving_avg
FROM employees;
```

05 Share of the department total

```
SELECT department_id, employee_name,
ROUND(100.0 * salary
  / SUM(salary) OVER (
    PARTITION BY department_id,
    1) AS pct_of_dept
FROM employees;
```

WATCH OUT

Default frame

With ORDER BY and no frame, the window is RANGE UNBOUNDED PRECEDING: tied rows share one running total. Write ROWS when you mean row by row.

LAST_VALUE

Under the default frame it returns the current row, or the last row tied with it. Add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

SECTION 18 · CTES & SUBQUERIES

CTEs & Subqueries.

03
PATTERNS

02
WATCH OUT

Break a query into named steps, compare each row with its group, and walk a hierarchy.

PATTERN

CODE

01 Name the steps with a CTE

```
WITH clean AS (
    SELECT employee_id, department_id, salary
    FROM employees
    WHERE salary > 0
),
by_dept AS (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM clean GROUP BY department_id
)
SELECT * FROM by_dept ORDER BY avg_salary DESC;
```

02 Above the department average

A correlated subquery; both optimisers rewrite it as a join

```
SELECT e.employee_name, e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(i.salary)
    FROM employees AS i
    WHERE i.department_id = e.department_id);
```

RESULT · WALK THE ORG CHART WITH A RECURSIVE CTE

```
WITH RECURSIVE chain AS (
    SELECT employee_id, employee_name, 0 AS depth
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.employee_name, c.depth + 1
    FROM employees AS e
    JOIN chain AS c ON e.manager_id = c.employee_id
)
SELECT depth, COUNT(*) AS people
FROM chain GROUP BY depth ORDER BY depth;
```

depth	people
0	1
1	7
2	11
3	1

Spark 4.1+ and DBR 17.0+. Fabric Warehouse does not support recursive queries: run it in Spark

SECTION 18 · CTES & SUBQUERIES

CONTINUED

PATTERN

CODE

03 Scalar subquery in SELECT

```
SELECT employee_name, salary,
       salary - (SELECT AVG(salary) FROM employees)
              AS vs_company_avg
FROM employees;
```

WATCH OUT

CTEs are not cached

Both engines usually inline a CTE, so it can run once per reference. When it is expensive and used often, write it to a table first.

Recursion limits

Databricks stops at 100 levels by default; raise it with `MAX RECURSION LEVEL n` after the CTE name.

SECTION 19 · PIVOT & UNPIVOT

Pivot & Unpivot.

03

PATTERNS

02

WATCH OUT

Turn rows into columns and back, with the built-in operators or a portable CASE.

01 Pivot regions into columns

T-SQL · FABRIC WAREHOUSE

```
SELECT *
FROM (SELECT d.department_name, d.region,
            e.salary
      FROM dbo.employees AS e
      JOIN dbo.departments AS d
            ON d.department_id = e.department_id) AS s
PIVOT (SUM(salary) FOR region
      IN ([EMEA], [APAC], [AMER])) AS p;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT *
FROM (SELECT d.department_name, d.region,
            e.salary
      FROM employees AS e
      JOIN departments AS d
            ON d.department_id = e.department_id)
PIVOT (SUM(salary) FOR region
      IN ('EMEA', 'APAC', 'AMER'));
```

SECTION 19 · PIVOT & UNPIVOT

CONTINUED

PATTERN

CODE

02 Portable pivot with CASE

Works on every engine and lets you mix different aggregates

```
SELECT e.department_id,
       SUM(CASE WHEN YEAR(e.hire_date) < 2020
                THEN 1 ELSE 0 END) AS before_2020,
       SUM(CASE WHEN YEAR(e.hire_date) >= 2020
                THEN 1 ELSE 0 END) AS from_2020
FROM employees AS e
GROUP BY e.department_id;
```

03 Unpivot columns into rows

T-SQL · FABRIC WAREHOUSE

```
SELECT department_id, metric, value
FROM dbo.dept_stats
UNPIVOT (value FOR metric
        IN (headcount, payroll)) AS u;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT department_id, metric, value
FROM dept_stats
UNPIVOT (value FOR metric
        IN (headcount, payroll));
```

WATCH OUT

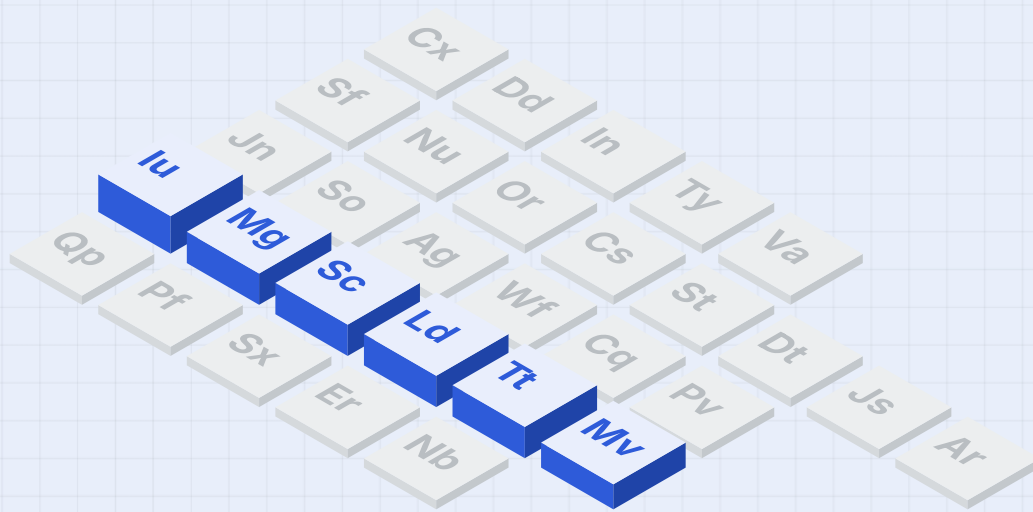
Values are fixed

PIVOT needs the list of values up front. A new region means editing the query, or dynamic SQL.

Unpivot drops NULL

Rows whose value is NULL disappear. Spark keeps them with `UNPIVOT INCLUDE NULLS`.

CHAPTER 04 OF 05



Change Data.

Insert, update and delete, upsert with MERGE, keep history with SCD, load files, travel back in time, and serve results as views.

06

SECTIONS

30

PATTERNS

10

WATCH OUT

IN THIS CHAPTER

20	Insert, Update, Delete	42
21	MERGE	43
22	SCD Type 1 & 2	45
23	Loading Data	47
24	Time Travel & History	49
25	Views & Materialised Views	50

SECTION 20 · INSERT, UPDATE, DELETE

Insert, Update, Delete.

06
PATTERNS

02
WATCH OUT

Change rows in place, overwrite a slice, and group several changes into one transaction.

PATTERN

CODE

01 Insert rows

```
INSERT INTO employees (employee_id, employee_name,
                        department_id, salary)
VALUES (1020, 'Ravi Kumar', 101, 99000);
```

02 Insert from a query

```
INSERT INTO eng
SELECT * FROM employees
WHERE department_id = 101
AND hire_date >= '2024-01-01';
```

03 Update from another table Delta UPDATE takes no FROM clause; MERGE does the joined version

T-SQL · FABRIC WAREHOUSE

```
UPDATE e
SET e.salary = u.salary
FROM dbo.employees AS e
JOIN dbo.employee_updates AS u
ON u.employee_id = e.employee_id;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
UPDATE employees
SET salary = 0
WHERE salary < 0;
-- a join-driven update is a MERGE
-- in Spark SQL: see section 21
```

04 Delete rows

```
DELETE FROM employees
WHERE salary < 0;
```

05 Replace a slice of a table Databricks, Delta

T-SQL · FABRIC WAREHOUSE

```
BEGIN TRAN;
DELETE FROM dbo.employees
WHERE hire_date >= '2026-01-01';
INSERT INTO dbo.employees
SELECT * FROM dbo.stage_employees;
COMMIT;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
INSERT INTO employees
REPLACE WHERE hire_date >= '2026-01-01'
SELECT * FROM stage_employees;
```

SECTION 20 · INSERT, UPDATE, DELETE

CONTINUED

06 Several changes in one transaction

Databricks

Databricks: GA since July 2026 for Unity Catalog managed Delta tables with catalog commits; no DDL inside

T-SQL · FABRIC WAREHOUSE

```
BEGIN TRAN;
UPDATE dbo.employees SET salary = 0
WHERE salary < 0;
DELETE FROM dbo.employees
WHERE employee_id = 1018;
COMMIT;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
BEGIN ATOMIC
  UPDATE employees SET salary = 0
  WHERE salary < 0;
  DELETE FROM employees
  WHERE employee_id = 1018;
END;
```

WATCH OUT

Rewrites

A Delta UPDATE or DELETE rewrites every file it touches, or marks rows with deletion vectors where enabled. Filter on clustering keys to touch fewer files.

Fabric conflicts

Fabric Warehouse locks at table level: two transactions writing the same table conflict even on different rows. The second one fails and must retry.

SECTION 21 · MERGE

MERGE.

05
PATTERNS

02
WATCH OUT

Upsert changes in one statement, update only what changed, and delete what the source no longer has.

THE STAGED CHANGES

EMPLOYEE_UPDATES · ALL 4 ROWS

employee_id	employee_name	job_title	department_id	salary	change_date
1002	Jeremy Roberts	Lead Engineer	101	124000	2026-04-01
1008	Tom Becker	Junior Engineer	101	64000	2026-04-01
1011	April Williams	Associate PM	106	84000	2026-04-01
1019	Leah Fischer	Security Engineer	108	112000	2026-04-01

A raise for 1002, a first salary for 1008, a fix for the -5000 of 1011, and 1019, a new starter. The MERGE and SCD patterns apply these to employees.

SECTION 21 • MERGE

CONTINUED

01 Upsert Fabric: GA; the closing semicolon is required, and USING cannot take VALUES

T-SQL • FABRIC WAREHOUSE

```
MERGE dbo.employees AS t
USING dbo.employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED THEN UPDATE SET
  t.job_title = s.job_title,
  t.salary    = s.salary
WHEN NOT MATCHED THEN INSERT
  (employee_id, employee_name,
   job_title, department_id, salary)
VALUES (s.employee_id, s.employee_name,
        s.job_title, s.department_id, s.salary);
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
MERGE INTO employees AS t
USING employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED THEN UPDATE SET
  job_title = s.job_title,
  salary    = s.salary
WHEN NOT MATCHED THEN INSERT
  (employee_id, employee_name,
   job_title, department_id, salary)
VALUES (s.employee_id, s.employee_name,
        s.job_title, s.department_id, s.salary);
```

02 Update only rows that changed

IS DISTINCT FROM also catches a NULL salary becoming a number. Same shape in T-SQL

```
MERGE INTO employees AS t
USING employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED
  AND t.salary IS DISTINCT FROM s.salary
THEN UPDATE SET salary = s.salary;
```

03 Delete what the source no longer has

DBR 12.2+

Same clause in T-SQL. Only run it with a complete source, never a daily increment

```
MERGE INTO employees AS t
USING current_staff AS s
  ON t.employee_id = s.employee_id
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

SECTION 21 · MERGE

CONTINUED

PATTERN

CODE

04 Update and insert every column Delta

```
MERGE INTO employees AS t
USING employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

05 Let new source columns into the target DBR 15.2+

```
MERGE WITH SCHEMA EVOLUTION INTO employees AS t
USING employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

WATCH OUT

Duplicate source keys

Two source rows for one target row fail the MERGE on both engines. Deduplicate the source first, as with employee 1007.

Merge condition

Put only the key in ON. Extra filters there turn matches into inserts and create duplicates.

SECTION 22 · SCD TYPE 1 & 2

SCD Type 1 & 2.

05
PATTERNS

01
WATCH OUT

Overwrite attributes in place, or keep every version with validity dates and a current flag.

PATTERN

CODE

01 Type 1: overwrite in place

Type 1 keeps no history: the old value is gone

```
MERGE INTO dim_employee AS t
USING employee_updates AS s
  ON t.employee_id = s.employee_id
WHEN MATCHED THEN UPDATE SET
  job_title = s.job_title;
```

SECTION 22 · SCD TYPE 1 & 2

CONTINUED

02 Type 2 dimension table

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.dim_employee (
  employee_key BIGINT IDENTITY,
  employee_id INT,
  job_title VARCHAR(100),
  salary DECIMAL(12,2),
  valid_from DATE,
  valid_to DATE,
  is_current BIT
);
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE TABLE dim_employee (
  employee_key BIGINT
    GENERATED ALWAYS AS IDENTITY,
  employee_id INT,
  job_title STRING,
  salary DECIMAL(12,2),
  valid_from DATE,
  valid_to DATE,
  is_current BOOLEAN
);
```

03 Type 2 in one MERGE

The NULL key never matches, so each changed row is closed and re-inserted as the new version. In T-SQL use 0 and 1 for the BIT flag

```
MERGE INTO dim_employee AS t
USING (
  SELECT s.employee_id AS merge_key, s.*
  FROM employee_updates AS s
  UNION ALL
  SELECT NULL AS merge_key, s.* -- changed rows, again
  FROM employee_updates AS s
  JOIN dim_employee AS d
    ON d.employee_id = s.employee_id AND d.is_current
    AND d.salary IS DISTINCT FROM s.salary
) AS src
ON t.employee_id = src.merge_key AND t.is_current
WHEN MATCHED AND t.salary IS DISTINCT FROM src.salary THEN
  UPDATE SET is_current = false,
             valid_to = src.change_date
WHEN NOT MATCHED THEN INSERT
  (employee_id, job_title, salary,
   valid_from, valid_to, is_current)
VALUES (src.employee_id, src.job_title, src.salary,
        src.change_date, NULL, true);
```

04 The version valid on a date

```
SELECT employee_id, job_title, salary
FROM dim_employee
WHERE DATE '2025-06-30' >= valid_from
      AND (DATE '2025-06-30' < valid_to OR valid_to IS NULL);
```

05 Let the pipeline do it

[Lakeflow pipelines](#)

```
CREATE FLOW employee_history AS AUTO CDC INTO dim_employee
FROM STREAM(employee_changes)
KEYS (employee_id)
SEQUENCE BY change_date
STORED AS SCD TYPE 2;
```

SECTION 22 · SCD TYPE 1 & 2

CONTINUED

WATCH OUT

One current row

Check with `GROUP BY employee_id HAVING SUM(CASE WHEN is_current THEN 1 ELSE 0 END) <> 1`: any row returned is a broken dimension.

SECTION 23 · LOADING DATA

Loading Data.

05
PATTERNS

02
WATCH OUT

Load files with COPY INTO, query files in place, create tables from files and load incrementally.

01 Load files into a table Fabric reads CSV, Parquet and JSONL; Databricks skips files it has already loaded

T-SQL · FABRIC WAREHOUSE

```
COPY INTO dbo.employees
FROM 'https://acct.dfs.core.windows.net/raw/emp/*.parquet'
WITH (FILE_TYPE = 'PARQUET');
```

SPARK SQL · DATABRICKS

```
COPY INTO employees
FROM '/Volumes/main/raw/files/emp/'
FILEFORMAT = PARQUET;
```

02 Query files without loading them

T-SQL · FABRIC WAREHOUSE

```
SELECT TOP 10 *
FROM OPENROWSET (
    BULK 'https://acct.dfs.core.windows.net/raw/emp/*.parquet'
) AS r;
```

SPARK SQL · DATABRICKS

```
SELECT *
FROM read_files('/Volumes/main/raw/files/emp/',
                format => 'csv', header => true)
LIMIT 10;
```

SECTION 23 · LOADING DATA

CONTINUED

PATTERN

CODE

03 Query files from any Spark [Spark SQL](#)

Works in Fabric Spark and open-source Spark; the path is relative to the lakehouse

```
SELECT * FROM parquet.`Files/raw/emp/`;
```

04 Create a table from files

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.raw_employees AS
SELECT *
FROM OPENROWSET (
    BULK 'https://acct.dfs.core.windows.net/raw/emp/*.parquet'
) AS r;
```

SPARK SQL · DATABRICKS

```
CREATE TABLE raw_employees AS
SELECT *
FROM read_files ('/Volumes/main/raw/files/emp/',
    format => 'parquet');
```

05 Load only what is new

A watermark on a load timestamp. Same on both engines

```
INSERT INTO silver_employees
SELECT *
FROM bronze_employees
WHERE ingested_at > (SELECT MAX(ingested_at)
    FROM silver_employees);
```

WATCH OUT

OneLake paths

COPY INTO from OneLake needs Entra ID and the workspace and item IDs in the URL.

Reloading

On Databricks, add `COPY_OPTIONS ('force' = 'true')` only when you mean to load the same files again.

SECTION 24 · TIME TRAVEL & HISTORY

Time Travel & History.

05
PATTERNS

01
WATCH OUT

Query a table as it was, copy an old version, undo a bad write and read the change feed.

01 Query the table as it was Fabric: UTC, at most three fractional digits, once per statement

T-SQL · FABRIC WAREHOUSE

```
SELECT *
FROM dbo.employees
OPTION (FOR TIMESTAMP AS OF
        '2026-09-01T10:00:00.000');
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT * FROM employees
TIMESTAMP AS OF '2026-09-01 10:00:00';

SELECT * FROM employees VERSION AS OF 12;
```

02 List the commits

Delta

```
DESCRIBE HISTORY employees;
```

03 Copy a table as it was A Fabric clone copies metadata only; a Delta DEEP CLONE copies the files

T-SQL · FABRIC WAREHOUSE

```
CREATE TABLE dbo.employees_0901
AS CLONE OF dbo.employees
AT '2026-09-01T10:00:00';
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE TABLE employees_0901
DEEP CLONE employees
TIMESTAMP AS OF '2026-09-01 10:00:00';
```

04 Undo a bad write

Delta

```
RESTORE TABLE employees TO VERSION AS OF 12;
```

In Fabric Warehouse, clone
the old version instead

SECTION 24 · TIME TRAVEL & HISTORY

CONTINUED

PATTERN

CODE

05 Read the change feed

Delta

```
ALTER TABLE employees
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

SELECT employee_id, _change_type, _commit_version
FROM table_changes('employees', 5);
```

WATCH OUT

Retention

Fabric keeps 30 days by default (1 to 120). Delta keeps log entries 30 days and removed files 7 days: after VACUUM, older versions cannot be read.

SECTION 25 · VIEWS & MATERIALISED VIEWS

Views & Materialised Views.

04

PATTERNS

02

WATCH OUT

Save a query as a view, persist results that refresh themselves, and stream files into a table.

01 Create or replace a view

T-SQL · FABRIC WAREHOUSE

```
CREATE OR ALTER VIEW dbo.v_payroll AS
SELECT department_id,
       SUM(salary) AS payroll
FROM dbo.employees
GROUP BY department_id;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE OR REPLACE VIEW v_payroll AS
SELECT department_id,
       SUM(salary) AS payroll
FROM employees
GROUP BY department_id;
```

02 Materialised view that refreshes daily

Databricks SQL

```
CREATE MATERIALIZED VIEW mv_payroll
SCHEDULE EVERY 1 DAY AS
SELECT department_id, SUM(salary) AS payroll
FROM employees
GROUP BY department_id;

REFRESH MATERIALIZED VIEW mv_payroll;
```

03 Materialised lake view in Fabric

Fabric Spark

Needs a schema-enabled lakehouse; refreshes incrementally when the sources allow it

```
CREATE OR REPLACE MATERIALIZED LAKE VIEW silver.payroll AS
SELECT department_id, SUM(salary) AS payroll
FROM bronze.employees
GROUP BY department_id;

REFRESH MATERIALIZED LAKE VIEW silver.payroll FULL;
```

SECTION 25 · VIEWS & MATERIALISED VIEWS

CONTINUED

PATTERN

CODE

04 Streaming table from a folder

Databricks SQL

```
CREATE OR REFRESH STREAMING TABLE bronze_employees AS
SELECT *
FROM STREAM read_files('/Volumes/main/raw/files/emp/',
                        format => 'csv');
```

WATCH OUT

Fabric Warehouse

No materialised views in the Warehouse. Persist results with CTAS on a schedule, or use a materialised lake view in the lakehouse.

Streaming tables

They expect append-only sources. An update or delete upstream breaks the stream until you handle it.

CHAPTER 05 OF 05



Tune, Secure & Operate.

Read plans, keep statistics and files healthy, lock data down, script logic and errors, and run SQL from notebooks and pipelines.

05

SECTIONS

22

PATTERNS

08

WATCH OUT

IN THIS CHAPTER

26	Query Plans	53
27	Performance & Maintenance	54
28	Security	55
29	Scripting & Errors	57
30	Notebooks & Pipelines	60

SECTION 26 · QUERY PLANS

Query Plans.

03
PATTERNS

01
WATCH OUT

See how a query will run, and find the slow ones after they have run.

01 Show the plan

T-SQL · FABRIC WAREHOUSE

```
SET SHOWPLAN_XML ON;
GO
SELECT department_id, COUNT(*)
FROM dbo.employees
GROUP BY department_id;
GO
SET SHOWPLAN_XML OFF;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
EXPLAIN FORMATTED
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

WHAT TO LOOK FOR IN A SPARK PLAN

IN THE PLAN	WHAT IT TELLS YOU
PushedFilters	The filter reached the file scan, so less data is read.
PartitionFilters	Whole folders were skipped. Empty on a filtered column means pruning did not happen.
Exchange	A shuffle: data moved across the network. Every extra one costs time.
BroadcastHashJoin	The small side was copied to every executor. SortMergeJoin means both sides were shuffled.

02 Slowest recent queries in Fabric

Fabric Warehouse

30 days of history; a new query can take up to 15 minutes to appear

```
SELECT TOP 10 start_time, total_elapsed_time_ms,
              command
FROM queryinsights.exec_requests_history
ORDER BY total_elapsed_time_ms DESC;
```

03 Slowest recent queries on Databricks

Databricks

```
SELECT statement_text, total_duration_ms
FROM system.query.history
ORDER BY total_duration_ms DESC
LIMIT 10;
```

WATCH OUT

Plan before run

EXPLAIN shows the plan before AQE changes it. The plan that actually ran is in the Spark UI SQL tab or the query profile.

SECTION 27 · PERFORMANCE & MAINTENANCE

Performance & Maintenance.

07
PATTERNS

02
WATCH OUT

Keep statistics fresh, compact and cluster files, clean up old versions and cache what is reused.

01 Refresh statistics Fabric creates statistics automatically; add them by hand after a big load

T-SQL · FABRIC WAREHOUSE

```
CREATE STATISTICS st_emp_dept
ON dbo.employees (department_id)
WITH FULLSCAN;
UPDATE STATISTICS dbo.employees;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
ANALYZE TABLE employees
COMPUTE STATISTICS FOR ALL COLUMNS;
```

02 Compact small files

Delta

```
OPTIMIZE employees;
```

03 Compact with V-Order in Fabric Fabric Spark

V-Order is off by default in new workspaces; turn it on for tables Power BI reads through Direct Lake

```
OPTIMIZE employees VORDER;
```

04 Cluster a table on its filter column Delta

Liquid clustering; on Databricks, CLUSTER BY AUTO picks the keys

```
ALTER TABLE employees CLUSTER BY (department_id);
OPTIMIZE employees;
```

05 Remove files older than 7 days Delta

```
VACUUM employees RETAIN 168 HOURS;
```

06 Cache results that are read often

Fabric result set caching is on by default; Databricks SQL warehouses also cache results automatically

T-SQL · FABRIC WAREHOUSE

```
ALTER DATABASE [HRWarehouse]
SET RESULT_SET_CACHING ON;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CACHE TABLE departments;
-- later
UNCACHE TABLE departments;
```

SECTION 27 · PERFORMANCE & MAINTENANCE

CONTINUED

PATTERN

CODE

07 Let Databricks maintain tables

Databricks

Runs OPTIMIZE, VACUUM and ANALYZE for Unity Catalog managed tables

```
ALTER SCHEMA hr ENABLE PREDICTIVE OPTIMIZATION;
```

WATCH OUT

VACUUM and time travel

VACUUM deletes the files old versions need. Never set retention below the longest query or time travel window you rely on.

Stale statistics

After a large load, row estimates are wrong and joins pick bad strategies. Refresh statistics as the last step of the load.

SECTION 28 · SECURITY

Security.

04

PATTERNS

01

WATCH OUT

Grant access, hide columns and rows from the wrong people, and mask sensitive values.

01 Grant read access Unity Catalog needs USE CATALOG and USE SCHEMA as well as SELECT

T-SQL · FABRIC WAREHOUSE

```
GRANT SELECT ON dbo.employees
TO [analyst@corp.com];
```

SPARK SQL · DATABRICKS

```
GRANT USE CATALOG ON CATALOG main TO `analysts`;
GRANT USE SCHEMA ON SCHEMA main.hr TO `analysts`;
GRANT SELECT ON TABLE main.hr.employees
TO `analysts`;
```

SECTION 28 • SECURITY

CONTINUED

02 Allow only some columns

T-SQL • FABRIC WAREHOUSE

```
GRANT SELECT ON dbo.employees
    (employee_id, employee_name, job_title)
TO [analyst@corp.com];
```

SPARK SQL • DATABRICKS

```
CREATE VIEW hr.v_employees_public AS
SELECT employee_id, employee_name, job_title
FROM hr.employees;
GRANT SELECT ON VIEW hr.v_employees_public
TO `analysts`;
```

03 Row-level security

T-SQL • FABRIC WAREHOUSE

```
CREATE FUNCTION sec.fn_region(@region VARCHAR(10))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS ok
WHERE @region = 'EMEA'
    OR USER_NAME() = 'hr.lead@corp.com';
GO
CREATE SECURITY POLICY sec.region_policy
ADD FILTER PREDICATE sec.fn_region(region)
ON dbo.departments
WITH (STATE = ON);
```

SPARK SQL • DATABRICKS

```
CREATE FUNCTION hr.region_filter(region STRING)
RETURN is_account_group_member('hr_global')
    OR region = 'EMEA';

ALTER TABLE departments
SET ROW FILTER hr.region_filter ON (region);
```

SECTION 28 • SECURITY

CONTINUED

04 Mask a sensitive column Fabric users need the UNMASK permission to see real values

T-SQL • FABRIC WAREHOUSE

```
ALTER TABLE dbo.employees
ALTER COLUMN email
ADD MASKED WITH (FUNCTION = 'email()');
```

SPARK SQL • DATABRICKS

```
CREATE FUNCTION hr.mask_email(email STRING)
RETURN CASE WHEN is_account_group_member('hr')
THEN email ELSE '***' END;

ALTER TABLE employees
ALTER COLUMN email SET MASK hr.mask_email;
```

WATCH OUT

Masking is not encryption

A masked column can still be filtered and grouped, so a determined user can infer values. Use column permissions for real secrets.

SECTION 29 • SCRIPTING & ERRORS

Scripting & Errors.

05
PATTERNS

02
WATCH OUT

Loop and branch in SQL, catch and raise errors, and wrap logic in procedures.

01 Loop with a counter Spark 4.1+

T-SQL • FABRIC WAREHOUSE

```
DECLARE @i INT = 1;
WHILE @i <= 3
BEGIN
    PRINT @i;
    SET @i = @i + 1;
END;
```

SPARK SQL • DATABRICKS, FABRIC SPARK

```
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 3 DO
        SET i = i + 1;
    END WHILE;
    SELECT i AS final_value;
END;
```

SECTION 29 · SCRIPTING & ERRORS

CONTINUED

02 Catch an error Spark 4.1+

T-SQL · FABRIC WAREHOUSE

```
BEGIN TRY
    SELECT 10 / 0;
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS code,
           ERROR_MESSAGE() AS msg;
END CATCH;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
BEGIN
    DECLARE EXIT HANDLER FOR DIVIDE_BY_ZERO
    BEGIN
        SELECT 'division failed' AS msg;
    END;
    SELECT 10 / 0;
END;
```

03 Raise your own error Databricks scripts can also use SIGNAL SQLSTATE '45000'

T-SQL · FABRIC WAREHOUSE

```
IF EXISTS (SELECT 1 FROM dbo.employees
           WHERE salary < 0)
    THROW 50001, 'Negative salary found', 1;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
SELECT raise_error('Negative salary found')
FROM employees
WHERE salary < 0
LIMIT 1;
```

SECTION 29 · SCRIPTING & ERRORS

CONTINUED

04 Wrap logic in a procedure DBR 17.0+, Unity Catalog

T-SQL · FABRIC WAREHOUSE

```
CREATE OR ALTER PROCEDURE dbo.give_raise
    @id INT, @pct DECIMAL(5,2)
AS
BEGIN
    UPDATE dbo.employees
    SET salary = salary * (1 + @pct / 100)
    WHERE employee_id = @id;
END;
GO
EXEC dbo.give_raise @id = 1002, @pct = 5;
```

SPARK SQL · DATABRICKS, FABRIC SPARK

```
CREATE PROCEDURE hr.give_raise(
    id INT, pct DECIMAL(5,2))
LANGUAGE SQL SQL SECURITY INVOKER
AS BEGIN
    UPDATE employees
    SET salary = salary * (1 + pct / 100)
    WHERE employee_id = id;
END;
CALL hr.give_raise(1002, 5);
```

05 Loop over a query result Spark 4.1+

```
BEGIN
    DECLARE total INT DEFAULT 0;
    FOR r AS SELECT department_id FROM departments DO
        SET total = total + 1;
    END FOR;
    SELECT total AS departments_seen;
END;
```

WATCH OUT

THROW and semicolons

The statement before THROW must end with a semicolon, or T-SQL reads THROW as an alias.

No cursors in Fabric

Fabric Warehouse has no cursors. Write set-based SQL, or a WHILE loop over a numbered temp table.

SECTION 30 · NOTEBOOKS & PIPELINES

Notebooks & Pipelines.

03
PATTERNS

02
WATCH OUT

Run SQL from notebook cells and Python, pass parameters, and know which engine each tool talks to.

01 A SQL cell in a notebook

FABRIC NOTEBOOK

```
%%sql
SELECT department_id, COUNT(*) AS n
FROM employees
GROUP BY department_id
```

DATABRICKS NOTEBOOK

```
%sql
SELECT department_id, COUNT(*) AS n
FROM employees
GROUP BY department_id
```

02 SQL from Python with a parameter

Spark 3.4+

Parameters are passed as values, never pasted into the SQL text

```
spark.sql(
    "SELECT * FROM employees WHERE salary > :min",
    args={"min": 100000},
).show()
```

03 Mix a DataFrame and SQL

PySpark

```
df = spark.table("employees").where("salary > 0")
df.createOrReplaceTempView("emp_clean")
spark.sql("SELECT department_id, AVG(salary) "
          "FROM emp_clean GROUP BY department_id").show()
```

WATCH OUT

%%sql is Spark SQL

In Fabric, a %%sql cell runs Spark SQL against the lakehouse. For T-SQL against a Warehouse, use a T-SQL notebook or the SQL query editor.

Pipelines

A Fabric pipeline Script activity sends T-SQL to the Warehouse. A Lakeflow Jobs SQL task runs a query or file on a SQL warehouse.

APPENDIX A · T-SQL TO SPARK SQL

35
TASKS

Translate between the dialects.

The same task in both dialects, one row each. The sections have the full patterns; this is the lookup when a query has to move from one engine to the other.

QUERY BASICS

TASK	T-SQL · FABRIC WAREHOUSE	SPARK SQL
Quote a name	[order id]	`order id`
First N rows	SELECT TOP 10 ...	... LIMIT 10
Skip rows, then take	ROW_NUMBER() + WHERE	LIMIT 10 OFFSET 20
Join text T-SQL CONCAT skips NULL; Spark returns NULL	a + b, CONCAT (a, b)	a b, concat (a, b)
Default for NULL Spark isnull() takes one argument	ISNULL (x, 0)	coalesce (x, 0)
Inline if	IIF (c, a, b)	if (c, a, b)
Cast without failing	TRY_CAST (x AS INT)	try_cast (x AS INT)
NULL-safe equals	a IS NOT DISTINCT FROM b	a <=> b

TEXT AND DATES

TASK	T-SQL · FABRIC WAREHOUSE	SPARK SQL
Length LEN ignores trailing spaces	LEN (s)	length (s)
Position of a character	CHARINDEX ('@', s)	instr (s, '@')
Split into rows	STRING_SPLIT (s, ',')	explode (split (s, ','))
Aggregate text Spark 4.0+	STRING_AGG (x, ',')	string_agg (x, ',')
Now	SYSUTCDATETIME ()	current_timestamp ()
Add months	DATEADD (month, 6, d)	add_months (d, 6)
Days between Arguments reversed	DATEDIFF (day, a, b)	datediff (b, a)
Truncate to month	DATETRUNC (month, d)	trunc (d, 'MM')
Read a JSON value VARIANT: j:a	JSON_VALUE (j, '\$.a')	get_json_object (j, '\$.a')

APPENDIX A · T-SQL TO SPARK SQL

CONTINUED

AGGREGATES AND WINDOWS

TASK	T-SQL · FABRIC WAREHOUSE	SPARK SQL
Median A window in T-SQL, an aggregate in Spark	PERCENTILE_CONT (0.5) ... OVER	median (x)
Approximate distinct	APPROX_COUNT_DISTINCT (x)	approx_count_distinct (x)
Filter on a window Spark: Databricks or Spark 4.2+	QUALIFY rn = 1	QUALIFY rn = 1
Recursive query Spark 4.1+, DBR 17.0+	not supported	WITH RECURSIVE
Unpivot Spark 3.4+	UNPIVOT (...)	UNPIVOT (...)

TABLES AND CHANGES

TASK	T-SQL · FABRIC WAREHOUSE	SPARK SQL
Scratch table	#t ... WITH (DISTRIBUTION = ROUND_ROBIN)	CREATE TEMP VIEW
Identity key	BIGINT IDENTITY	GENERATED ALWAYS AS IDENTITY
Upsert The T-SQL semicolon is required	MERGE ... ;	MERGE INTO ...
Update from a join	UPDATE ... FROM ... JOIN	MERGE INTO
Table as it was	OPTION (FOR TIMESTAMP AS OF '...')	TIMESTAMP AS OF '...'
Copy a table cheaply Databricks	CREATE TABLE ... AS CLONE OF	SHALLOW CLONE
Load files Different options on each	COPY INTO ... FROM	COPY INTO ... FROM

APPENDIX A · T-SQL TO SPARK SQL

CONTINUED

SCRIPTING AND OPERATIONS

TASK	T-SQL · FABRIC WAREHOUSE	SPARK SQL
Variable	<code>DECLARE @x INT = 1</code>	<code>DECLARE VARIABLE x INT DEFAULT 1</code>
Dynamic table name	<code>EXEC (... QUOTENAME (@t))</code>	<code>IDENTIFIER(t)</code>
Catch an error Spark 4.1+	<code>BEGIN TRY ... BEGIN CATCH</code>	<code>DECLARE EXIT HANDLER</code>
Raise an error	<code>THROW 50001, 'msg', 1</code>	<code>raise_error('msg')</code>
Refresh statistics	<code>UPDATE STATISTICS t</code>	<code>ANALYZE TABLE t COMPUTE STATISTICS</code>
Show the plan	<code>SET SHOWPLAN_XML ON</code>	<code>EXPLAIN FORMATTED</code>

APPENDIX B · COMMON ERRORS

Common errors and fixes.

15

ERRORS

Each error below was triggered on Spark 4.1 with the dataset, and each fix was run to confirm it works. Many are T-SQL habits carried into Spark SQL. Messages are shortened.

ERROR	WHY IT HAPPENS	FIX
[PARSE_SYNTAX_ERROR] Syntax error at or near '5'. <code>SELECT TOP 5 * FROM employees</code>	T-SQL habits: TOP, [brackets] and + for text do not parse in Spark SQL.	<code>SELECT `employee_name` FROM employees LIMIT 5;</code>
[UNRESOLVED_ROUTINE] Cannot resolve routine `GETDATE` on search path [`system`.`builtin`, `system`.`session`, `spark_catalog`.`default`]. <code>SELECT GETDATE()</code>	A T-SQL function with no Spark equivalent of the same name.	<code>SELECT current_timestamp();</code>
[WRONG_NUM_ARGS.WITHOUT_SUGGESTION] The `isnull` requires 1 parameters but the actual number is 2. Please, refer to... <code>SELECT ISNULL(salary, 0) FROM employees</code>	Spark has isnull, but it tests one value and returns true or false.	<code>SELECT coalesce(salary, 0) FROM employees;</code>

APPENDIX B · COMMON ERRORS

CONTINUED

ERROR	WHY IT HAPPENS	FIX
[DATATYPE_MISMATCH.BINARY_OP_WRONG_TYPE] Cannot resolve "(employee_name + ())" due to data type mismatch: the binary operator requires the input type... <pre>SELECT employee_name + ' (' + job_title + ')' FROM employees</pre>	In Spark, + is arithmetic only.	<pre>SELECT employee_name ' (' job_title ')' FROM employees;</pre>
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function parameter with name `salary` cannot be resolved. Did you mean one of the following?... <pre>SELECT salary FROM employees</pre>	A typo, or a column dropped earlier. Spark lists the closest names.	<pre>DESCRIBE TABLE employees;</pre>
[AMBIGUOUS_REFERENCE] Reference `department_id` is ambiguous, could be: ['d`.`department_id`, 'e`.`department_id`]. <pre>SELECT department_id FROM employees e JOIN departments d ON ...</pre>	The column exists on both sides of the join.	<pre>SELECT e.department_id FROM employees AS e JOIN departments AS d ON e.department_id = d.department_id;</pre>
[MISSING_AGGREGATION] The non-aggregating expression "employee_name" is based on columns which are not participating in the GROUP BY clause. <pre>SELECT department_id, employee_name, COUNT(*) ... GROUP BY department_id</pre>	Every selected column must be grouped or aggregated.	<pre>SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;</pre>
[_LEGACY_ERROR_TEMP_1034] It is not allowed to use window functions inside WHERE clause. <pre>... WHERE ROW_NUMBER() OVER (...) = 1</pre>	WHERE runs before window functions are computed.	<pre>SELECT * FROM (SELECT *, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn FROM employees) WHERE rn = 1;</pre>
[CAST_INVALID_INPUT] The value 'Grace Okafor' of the type "STRING" cannot be cast to "INT" because it is malformed. Correct the value... <pre>SELECT CAST(employee_name AS INT) FROM employees</pre>	ANSI mode, the Spark 4 default, refuses a value that cannot be converted.	<pre>SELECT try_cast(employee_name AS INT) FROM employees;</pre>

APPENDIX B · COMMON ERRORS

CONTINUED

ERROR	WHY IT HAPPENS	FIX
[DIVIDE_BY_ZERO] Division by zero. Use `try_divide` to tolerate divisor being 0 and return NULL instead. If necessary set... <pre>SELECT salary / 0 FROM employees</pre>	ANSI mode raises on division by zero instead of returning NULL.	<pre>SELECT try_divide(salary, 0) FROM employees;</pre>
[ARITHMETIC_OVERFLOW] integer overflow. Use 'try_multiply' to tolerate overflow and return NULL instead. If necessary set... <pre>SELECT salary * 100000 FROM employees</pre>	The result is too big for INT.	<pre>SELECT CAST(salary AS BIGINT) * 100000 FROM employees;</pre>
[CANNOT_PARSE_TIMESTAMP] Text '30/09/2026' could not be parsed at index 0. Use `try_to_date` to tolerate invalid input string and return... <pre>SELECT to_date('30/09/2026', 'yyyy-MM-dd')</pre>	The pattern does not match the text.	<pre>SELECT to_date('30/09/2026', 'dd/MM/yyyy');</pre>
[NUM_COLUMNS_MISMATCH] UNION can only be performed on inputs with the same number of columns, but the first input has 2 columns and the... <pre>SELECT employee_id, employee_name ... UNION ALL SELECT department_id ...</pre>	Both sides of a set operation need the same number of columns.	<pre>SELECT employee_id, employee_name FROM employees UNION ALL SELECT department_id, department_name FROM departments;</pre>
[INVALID_TEMP_OBJ_REFERENCE] Cannot create the persistent object `spark_catalog`.`default`.`v_bad` of the type VIEW because it references to... <pre>CREATE VIEW v_bad AS SELECT * FROM employees</pre>	A saved view cannot read a temp view, which disappears with the session.	<pre>CREATE OR REPLACE TEMP VIEW v_ok AS SELECT * FROM employees;</pre>
[TABLE_OR_VIEW_NOT_FOUND] The table or view `employee` cannot be found. Verify the spelling and correctness of the schema and catalog. <pre>SELECT * FROM employee</pre>	Wrong name, or the wrong current catalog and schema.	<pre>SELECT current_catalog(), current_schema(); SHOW TABLES;</pre>

CHANGE NOTES

What changed since the SQL Handbook.

30
SECTIONS

142
PATTERNS

Compared with The SQL Handbook for Fabric & Databricks: what is new, what changed, what moved, and what the earlier edition got wrong.

NEW

Two books	The SQL Handbook is now two books: this cheat sheet for patterns, and The SQL Practitioner's Handbook 4.0 for how the engines think
One dataset	every pattern queries the same employees and departments tables, messy on purpose, with real result tables for counts, joins, ranking and the org chart
Tested	Spark SQL run on Spark 4.1 and T-SQL parsed; Delta, Unity Catalog and Fabric features checked against the documentation
7 new sections	Context & Names, Variables & Parameters, JSON & Semi-structured, Arrays & Maps, Loading Data, Views & Materialised Views, Scripting & Errors
2026 features	QUALIFY, VARIANT, SQL scripting, recursive CTEs in Spark, multi-statement transactions, Fabric Warehouse time travel and clones, materialised lake views, AUTO CDC

CHANGED

DLT	now Lakeflow Declarative Pipelines: <code>CREATE OR REFRESH STREAMING TABLE, read_files()</code> instead of <code>cloud_files()</code> , no <code>LIVE.</code> prefix
ZORDER	liquid clustering with <code>CLUSTER BY</code> is the default advice; <code>CLUSTER BY AUTO</code> on Databricks
Notebook parameters	named markers such as <code>:min_salary</code> instead of <code>\${region}</code>

MOVED TO THE HANDBOOK

Concepts	Delta architecture, choosing an engine, SQL or PySpark, decision guides and the glossary
-----------------	--

CORRECTED

Recursive CTEs	they arrived in DBR 17.0 and Spark 4.1, not DBR 9.1, and Fabric Warehouse does not support them
PIVOT in Spark	Spark SQL has had PIVOT since 2.4; CASE is only needed for portability
NOT MATCHED BY SOURCE	Fabric Warehouse MERGE supports it; no separate DELETE is needed
nvarchar	Fabric Warehouse tables have no nvarchar type; varchar is UTF-8



LAST WORD

Know which engine runs your query.

Across 30 sections and 142 patterns the same habits repeat: name the engine, handle NULL on purpose, deduplicate before you aggregate, and put only the key in a MERGE condition. The fastest fix is usually the one already written down.

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn



A NOTE ON THIS RELEASE

Version 4.0. T-SQL for Fabric Warehouse and Spark SQL for Databricks and Fabric Spark, tested on Spark 4.1, with Delta, Unity Catalog and Fabric features marked. Spotted an inaccuracy? Feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this cheat sheet together.

CONNECT

www.ssanjaychandra.com
linkedin.com/in/ssanjaychandra
ssanjaychandra@outlook.com